BEST VALUE HOTEL SELECTION

USING GREEDY ALGORITHM

Presented By - Batch 4

- Sudheksha S - 241365
- Indhumathi R - 241319
- Janagadevi Y - 241320
- Samiksa Sree S - 241352
- Joshna - 251463

PROBLEM MODELLING

Problem Statement:

Identify hotels that offer the best combination of price and distance from a venue by eliminating dominated hotels.

Input:

- N hotels, each having:
  - Price (₹) – Lower is better.
  - Distance (km) – Lower is better.

Output:

- Set of all non-dominated hotels (Skyline).

Objective:

- Identify every hotel that is not dominated by any other hotel.

Constraints:

- Maximum listings: 2×10^6.
- Listings are re-ranked every 5 minutes.
- Two attributes: Price and Distance.

Problem Classification:

- Enumeration Problem – Finds all non-dominated hotels.

Classical Problem Mapping:

- 2D Skyline Query / Pareto Frontier Problem

Dominance Example

Hotel	Price	Distance
A	Rs.2000	1 km
B	Rs.3000	3 km
C	Rs.2500	5 km

Dominance Rule:

Hotel X dominates Hotel Y if:

- X.price ≤ Y.price
- X.distance ≤ Y.distance
- At least one inequality is strict.

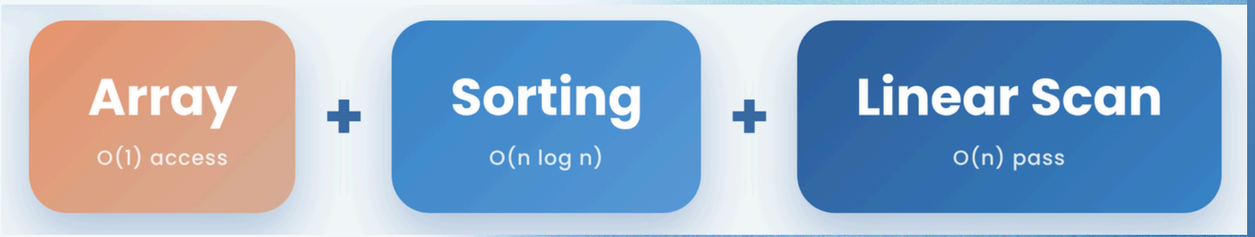
Result: Hotel A dominates both B and C. Only A remains in the Skyline.

Data Structure Analysis

Data Structure	Key Operations and Cost	Space	Behaviour at 2×10^6 Listings
Array (Chosen)	Access: O(1), Search: O(n), Sort: O(n log n)	O(n)	Efficient sorting and sequential scanning.
Linked List	Search: O(n), Insert/Delete: O(1)*, Sort: O(n log n)	O(n)	Pointer overhead and poor random access.
Balanced BST	Search/Insert/Delete: O(log n), Traversal: O(n)	O(n)	Supports dynamic updates but requires tree balancing.

Chosen Data Structure: **Array of Hotel Records**

- Efficient sorting by price.
- O(1) indexed access.
- Simple sequential traversal.
- Better cache locality.
- Suitable for large-scale batch processing.



CHOSEN TECHNIQUE: GREEDY ALGORITHM

Why Greedy?

01 The problem requires finding hotels that are not dominated by other hotels.
A hotel Hi dominates hotel Hj if:

1. Hi has a price less than or equal to Hj.
2. Hi has a distance less than or equal to Hj.
3. At least one of these values is strictly smaller.

Both price and distance must be minimized.

Greedy Strategy

02 We use a Sort-and-Sweep Greedy Algorithm.

- First, sort all hotels by price in ascending order.
- Traverse the sorted hotels from lowest to highest price.
- Maintain the minimum distance encountered so far.
- If the current hotel's distance is smaller than the minimum distance, select it.
- Otherwise, reject it because it is dominated by a previously selected hotel

03

Greedy-Choice Property

- The greedy-choice property states that a locally optimal choice can lead to a globally optimal solution.
- In this problem, selecting a hotel with a new minimum distance ensures that it cannot be dominated by any previously processed hotel.
- Since the hotels are processed in increasing price order, any hotel processed later cannot dominate an earlier hotel.
- Therefore, the greedy strategy correctly identifies the skyline.

04

Why Does It Work?

- After sorting by price, the only condition we need to check is whether the current distance is strictly smaller than all previously encountered distances.
- This allows us to solve the problem using a single traversal after sorting.

WHY OTHER TECHNIQUES DO NOT FIT

01 Divide and Conquer

Dividing hotels into smaller groups requires merging their skyline results. This adds unnecessary complexity compared to a single sort-and-sweep.

02 Dynamic Programming

No overlapping subproblems or recurrence relations exist. A single variable is enough to track the minimum distance.

03 Backtracking

Exploring different hotel combinations is computationally expensive for up to 2 million listings.

04 Branch and Bound

Requires a search space and bounding conditions. The problem can be solved directly using sorting and a greedy selection rule.

Greedy is preferred because it finds the **Pareto Frontier** efficiently in $O(n \log n)$ time without unnecessary search or computation.

Algorithm: Sort-and-Sweep Greedy

Algorithm Definition

Input: A set of hotels, each represented by price and distance.

Output: The set of non-dominated hotels sorted by increasing price.

Greedy Selection Rule

Select a hotel if and only if its distance is strictly smaller than the minimum distance encountered so far.

Pseudocode

```
Algorithm SkylineHotels(H):

    Sort H by price in ascending order

    skyline = empty list
    minDistance = infinity

    for each hotel h in H:

        if h.distance < minDistance:

            Add h to skyline
            minDistance = h.distance

        else:

            Reject h

    return skyline
```

Step by Step Algorithm

- Read all hotel listings.
- Sort the hotels by increasing price.
- Initialize minDistance to infinity.
- Traverse the sorted list.
- If the current distance is less than minDistance:
- Add the hotel to the skyline.
- Update minDistance.
- Otherwise, reject the hotel.
- Return the selected hotels.

Proof of Correctness – Greedy Sort-and-Sweep

1. Loop Invariant

- Hotels are sorted in ascending order of price.
- Maintain minDistance, which stores the minimum distance encountered so far.
- A hotel is selected only if its distance is strictly smaller than minDistance.

2. Correctness Argument

- If the current distance ≥ minDistance, a previously selected hotel has both lower price and lower or equal distance. Hence, REJECT.
- If the current distance < minDistance, no previously processed hotel can dominate it. Hence, KEEP.
- Future hotels have higher or equal prices, so they cannot dominate the selected hotel.

3. Conclusion

- Every selected hotel is non-dominated, and every rejected hotel is dominated by a previously selected hotel.
- Therefore, the Greedy Sort-and-Sweep algorithm correctly identifies the 2D Skyline (Pareto Frontier).

Step-by-Step Execution of the Greedy Algorithm

Trace on the Given Instance – Step-by-Step Execution

1 Original Hotel Data

Hotel	Price (₹)	Distance (km)
H1	4200	1.9
H2	3100	2.5
H3	5600	0.4
H4	2800	4.0
H5	3900	1.8
H6	2500	5.5
H7	4600	0.9
H8	3300	2.8
H9	6100	0.3
H10	3000	4.2

2 Sorted Order (Ascending Price)

H6 → H4 → H10 → H2 → H8 → H5 → H1 → H7 → H3 → H9

3 Greedy Selection – Iteration Table

Step	Hotel	Distance (km)	Min Distance Before (km)	Decision
1	H6	5.5	∞	KEEP
2	H4	4.0	5.5	KEEP
3	H10	4.2	4.0	REJECT
4	H2	2.5	4.0	KEEP
5	H8	2.8	2.5	REJECT
6	H5	1.8	2.5	KEEP
7	H1	1.9	1.8	REJECT
8	H7	0.9	1.8	KEEP
9	H3	0.4	0.9	KEEP
10	H9	0.3	0.4	KEEP

4 Final Skyline

Final Pareto Frontier

{H6, H4, H2, H5, H7, H3, H9}

Time and Space Complexity

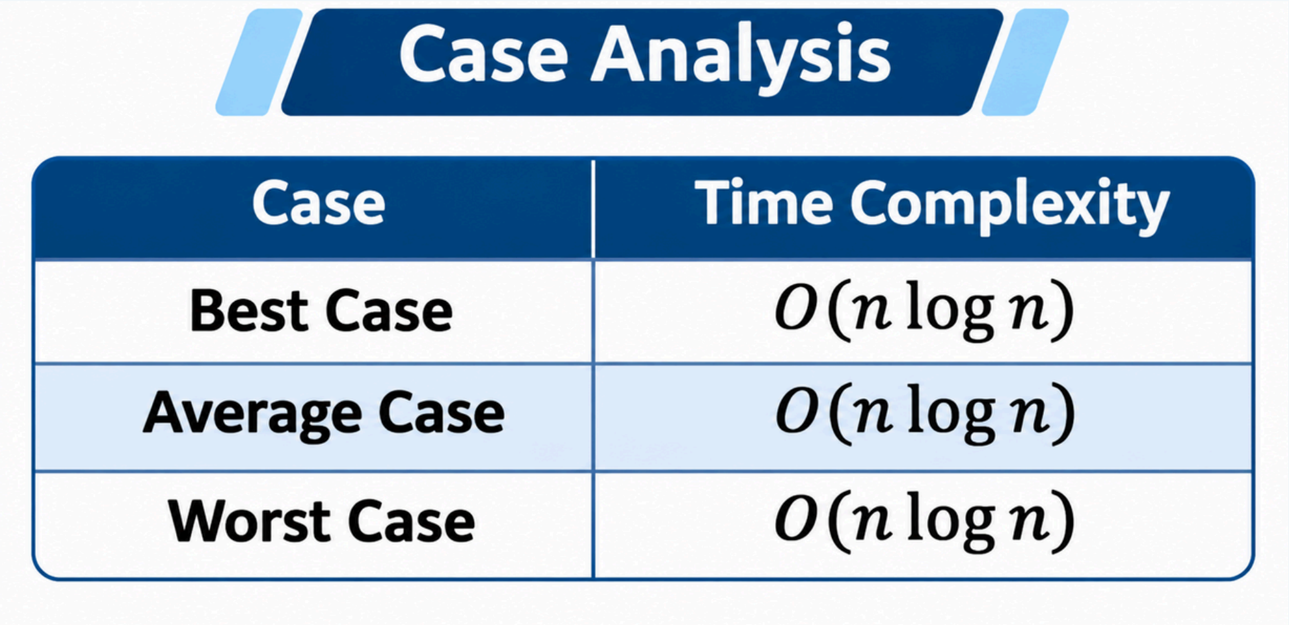
Let n be the number of hotel listings.

1. Operation-wise Complexity

Operation-wise Complexity

Operation	Time Complexity	Space Complexity
Sorting hotels by price	$O(n \log n)$	$O(n)$
Sweeping through sorted hotels	$O(n)$	$O(1)$
Total Complexity	$\boldsymbol{O(n \log n)}$	$\boldsymbol{O(n)}$

2. Case Analysis



3. Concrete Evidence

- Maximum hotel listings = 2×10^6.
- Comparing every hotel with every other hotel requires approximately:

$$\frac{n(n-1)}{2} \sim 2x10\text{^}2$$

Greedy Sort-and-Sweep requires:

- Sorting: O(n log n)
- Traversal: O(n)

Worst-Case Input

Worst Case: Every hotel is non-dominated

Observations

- Price increase
- Distance decreases.
- No hotel dominates another.
- All 8 hotels are retained.

Complexity

- Sorting: O(n log n)
- Sweeping: O(n)
- Overall Time Complexity: O(n log n)
- Output Space: O(n) in the worst case.

Worst Case: Every hotel is non-dominated

Hotel	Price (₹)	Distance (km)
H1	1,000	5.0
H2	2,000	4.5
H3	3,000	4.0
H4	4,000	3.5
H5	5,000	3.0
H6	6,000	2.5
H7	7,000	2.0
H8	8,000	1.5

Skyline Graph

Key Takeaways:

- Price increases, Distance decreases
- No hotel dominates another
- Time Complexity: O(n log n)

The worst case occurs when every hotel is non-dominated. The Sort-and-Sweep algorithm still efficiently identifies all skyline hotels with O(n log n) time complexity.

Implementation & Empirical Analysis

Programming Language: Java

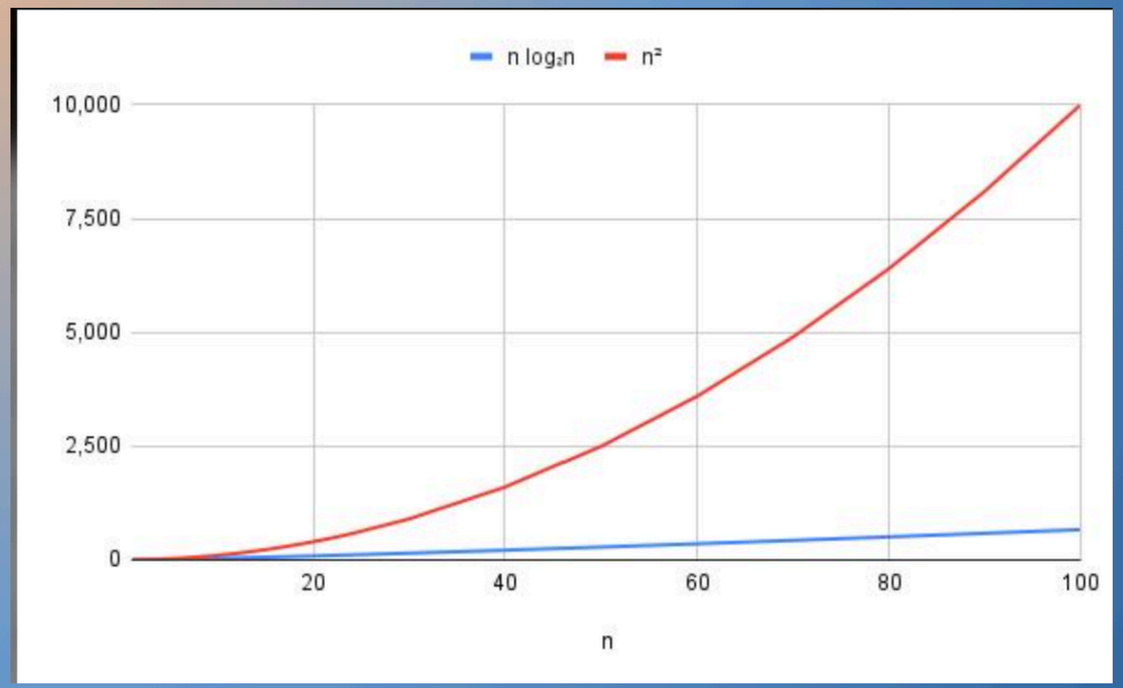
Algorithms Compared:

- Brute Force: Checks every hotel against every other hotel.
- Sort-and-Sweep: Sorts hotels and scans them once.

The hotel records are stored in an array containing price and distance.

Execution Time: Measured using System.nanoTime().

Parameter	Brute Force	Sort-and-Sweep
Sorting	Not required	Required
Comparison	Every pair	One scan after sorting
Time Complexity	$O(n^2)$	O(n log n)
Scalability	Low	High



Experimental Input Sizes

100 | 500 | 1,000 | 5,000 | 10,000 hotels

- Brute Force requires approximately quadratic growth in work.
- Sort-and-Sweep requires O(n log n) work.
- Sort-and-Sweep is more suitable for large hotel datasets.

Experimental analysis demonstrates the efficiency of Sort-and-Sweep compared with the Brute Force approach, particularly as the number of hotel listings increases.

Changed Requirement:

New Attribute: Guest Rating (Higher is Better)

Original: Price + Distance

Modified: Price + Distance + Rating

New Dominance Condition

Hotel A dominates Hotel B if:

- A's price is less than or equal to B's price.
- A's distance is less than or equal to B's distance.
- A's guest rating is greater than or equal to B's rating.
- At least one condition is strictly better.

- The original algorithm maintains only the minimum distance.
- Guest rating introduces another dimension.
- A hotel with a greater distance may have a better rating.
- A single minimum-distance value is no longer sufficient to identify all non-dominated hotels.

Modified Approach

A three-dimensional skyline algorithm is required.

Possible approaches include:

- Divide-and-Conquer Skyline Algorithm.
- Multidimensional Skyline Processing.
- Brute Force Dominance Checking for smaller datasets.

Distributed Hotel Selection Across 16 Servers

Scenario: Hotel information is distributed across 16 different servers instead of being stored in a single database.

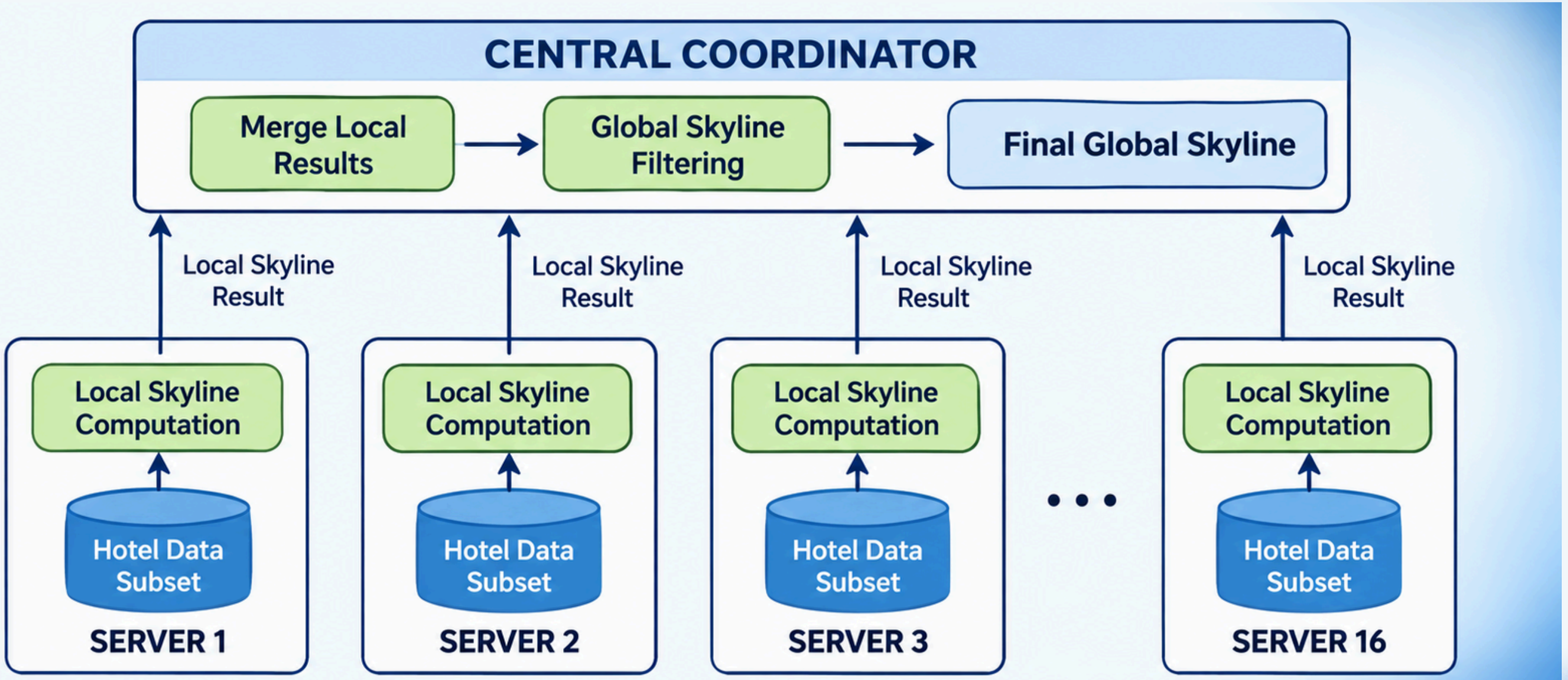
Problem:

- Each server contains information about a subset of hotels.
- Need to identify the best-value hotels from all 16 servers.
- Comparing every hotel globally can increase communication and processing costs.

Proposed Solution: Distributed Skyline Computation

- Divide the hotel dataset among 16 servers.
- Each server independently computes its local skyline.
- Send local skyline results to a central coordinator.
- Merge the local skyline results.
- Perform global dominance checking.
- Obtain the final global skyline of hotels.

Distributed Skyline Architecture



Hotel records are partitioned across 16 servers; local skylines are merged and globally filtered.

- **Data Distribution:**
  Hotel records are divided among 16 servers.
- **Local Processing:**
  Each server applies skyline computation to its local records.
- **Result Transmission:**
  Local skyline results are sent to the coordinator.
- **Merging:**
  The coordinator combines the local skyline results.
- **Global Filtering:**
  Dominated hotels are eliminated.
- **Final Output:**
  The remaining hotels form the global skyline.

Advantages:

- Parallel processing.
- Reduced local processing workload.
- Scalability.
- Suitable for large distributed databases.

Single Sort-and-Sweep vs Distributed Skyline

Parameter	Single Sort-and-Sweep	Distributed Skyline
Data storage	Single database	16 servers
Processing	One system	Multiple systems
Computation	Sorting and scanning	Local skyline and merging
Communication	Minimal	Server communication required
Scalability	Limited by single system resources	Supports distributed processing
Output	Global skyline	Global skyline

Real-World Research Reference

Research Paper:
An Efficient Approach to Finding Potential Products Continuously
Research Details:

- Authors: Yu-Ling Hsueh, He Ma, Chia-Chun Lin and Roger Zimmermann.
- Journal: Information Systems.
- Volume: 65, 2017.
- DOI: 10.1016/j.is.2016.10.003.

Application to Hotel Selection:

- Customers consider multiple attributes when selecting hotels.
- Common attributes include price, distance and customer rating.
- Skyline queries identify non-dominated hotels.
- Hotels dominated by other hotels are eliminated.
- The final result provides multiple competitive hotel choices.

Illustrative Example:

HOTEL	PRICE($)	DISTANCE(MILES)	RATING
P1	209	0.1	3
P2	89	0.8	4

Observation: P1 offers a shorter distance, whereas P2 offers a lower price and higher rating. Neither dominates the other.

Real-World Applications

Domain	Application
Hotel booking	Comparing price and guest rating
Travel platforms	Selecting hotels based on budget and location
E-commerce	Comparing product price and ratings
Real estate	Comparing property price and location
Service platforms	Comparing service cost and quality

Project Contribution:

- Uses a greedy Sort-and-Sweep technique.
- Sorts hotel records based on selected criteria.
- Applies dominance checking.
- Identifies the Pareto Frontier.
- Provides multiple non-dominated hotel choices.

Conclusion:

Greedy-based Sort-and-Sweep efficiently identifies non-dominated hotels using multiple selection criteria. It helps customers make informed decisions without depending on a single scoring formula.

THANK YOU